UNIVERZITET U NOVOM SADU

FAKULTET TEHNIČKIH NAUKA U NOVOM SADU

Seminarski rad iz predmeta

Automatizacija upravljanja proizvodnim sistemima

Profesor: dr Darko Stefanović ***Student: Biljana Bečić I2 100/2020***

Novi Sad, 2021.

Sadržaj

1. Uvod ..... - 1 -
2. Opis studije slučaja ..... - 1 -
3. Korišćene tehnologije ..... - 2 -
3.1. Microsoft SQL Server Management Studio ..... - 2 -
3.2. Visual Studio ..... - 2 -
3.3. WCF & WPF ..... - 2 -
3.4. API ..... - 2 -
4. Prikaz rešenja ..... - 3 -
4.1. WPF & WCF ..... - 4 -
4.2. Implementacija API-ja ..... - 8 -
5. Zaključak ..... - 10 -
6. Literatura ..... - 11 -

1. Uvod

Današnja poslovanja je nemoguće zamisliti bez savremenih aplikacija koje predstavljaju podršku i unapređenje procesima koji se odvijaju unutar svakog sistema. Realizacija aplikacije koja bi podržala evidentiranje napravljenih rezervacija u okviru pozorišta predstavlja primarni cilj rada. Pregled i kratak opis upotrebljenih tehnologija za razvoj aplikacije, ključni koraci pri izradi aplikacije koji podrazumevaju kreiranje baze podataka, nakon toga servisa i same aplikacije kao i krajnji rezultat prethodnih koraka su prikazani u radu. Nakon toga se na glavnu formu aplikacije integrišu API-ji koji upotpunjuju aplikaciju i na kraju je prikazan izgled glavne forme sa svim funkcionalnostima kao i predlozima za unapređenja sadašnje verzije aplikacije.

2. Opis studije slučaja

Smanjenje nivoa grešaka i gubitaka u poslovanju je jedan od glavnih razloga primene informacionih sistema za skladištenje potrebnih podataka. Proces rezervacije karata unutar pozorišta bio bi olakšan i ubrzan uz pomoć odgovarajućeg sistema koji bi sadržao sve potrebne infromacije za sve entitete uključene u proces rezervacije karata. Glavni entiteti u sistemu pozorište pri procesu rezervaciju karata predstavljaju blagajnik (*Booking Clerk*), rezervacija (*Booking*), posetilac(*Visitor*), izvođač(*Performer*), predstava(*Play*), karta(*Ticket*), sala(*Venue*) i sedište(*Seat*).

Pri procesu rezervacije potrebni su podaci o posetiocu, karti koja je izdata, blagajniku koji je izdao kartu posetiocu i datumu rezervacije. Pri rezervaciji se od posetioca traži broj telefona ili email adresa, ukoliko dođe do promene rezervacije da postoji mogućnost kontakta sa posetiocem, dok se karta koja se izdaje vezuje za jednu predstavu i ima određenu cenu. Dalje svaka ta predstava ima definisan datum izvedbe predstave, naziv predstave kao i trajanje iste u minutama takođe je predstava vezana za salu u kojoj se izvodi. Izvođač je vezan za određenu predstavu i beleži se ime izvođača, prezime i pol. U evidenciji sala postoji naziv svake sale kao i kapacitet sedišta, što predstavlja kontigent karata za određenu predstavu. Što se tiče sedišta evidentira se u kojoj sali se nalazi i tačan broj reda i broj kolone u kojoj se nalazi to sedište.

3. Korišćene tehnologije

3.1. Microsoft SQL Server Management Studio

SQL Server Management Studio je softver koji služi za konekciju sa *SQL Server*-om i za izvršavanje operacija na istom. *Management Studio* je besplatan softver i može biti preuzet sa *Microsoft website*-a. Obezbeđuje alate za konfigurisanje, nadgledanje i administriranje. Podržava *Structured Query Language* (SQL), klijent/server relacionu bazu podataka.

3.2. Visual Studio

Visual Studio predstavlja integrisano razvojno okruženje (*Integrated Development Environment*), proizvod *Microsoft*-a koji služi za razvijanje grafičkog korisničkog interfejsa (*Graphical User Interface*), *web* aplikacija, mobilnih aplikacija, cloud-a i drugih *web* usluga. Nije specificiran jezik koji mora da se koristi tako da kod može da bude napisan u *C#, C++, Visual Basic, Python, JavaScript* kao i mnogim drugim programskim jezicima, ukupno 36 različitih programskih jezika podržava.

3.3. WCF & WPF

Windows Communication Foundation (WCF) je radni okvir za izgradnju servisno-orijentisanih aplikacija. Postoji mogućnost slanja podataka u obliku asinhronih poruka s jedne servisne krajnje tačke na drugu. Krajnja tačka servisa može biti deo stalno dostupne usluge čiji je domaćin IIS, ili može biti servis u aplikaciji. Krajnja tačka može biti klijent usluge koji traži podatke sa krajnje tačke usluge. Poruke mogu biti jednostavne poput jednog znaka ili reči poslate kao XML ili složenije [1].

Windows Presentation Foundation (WPF) je radni okvir za kreiranje *Windows* aplikacija, grafički podsistem koji služi za prikazivanje korisničkog interfejsa u aplikacijama, takođe pruža mogućnost izrade desktop ili web aplikacija. WPF sadrži niz funkcija za razvoj aplikacija koje uključuju *Extensible Application Markup Language* (XAML), kontrole, *data binding*, *layout*, 2D i 3D grafiku, animaciju i mnoge druge. WPF je deo .NET-a, tako da postoji mogućnost da pravite aplikacije koje sadrže druge elemente .NET API-ja [2].

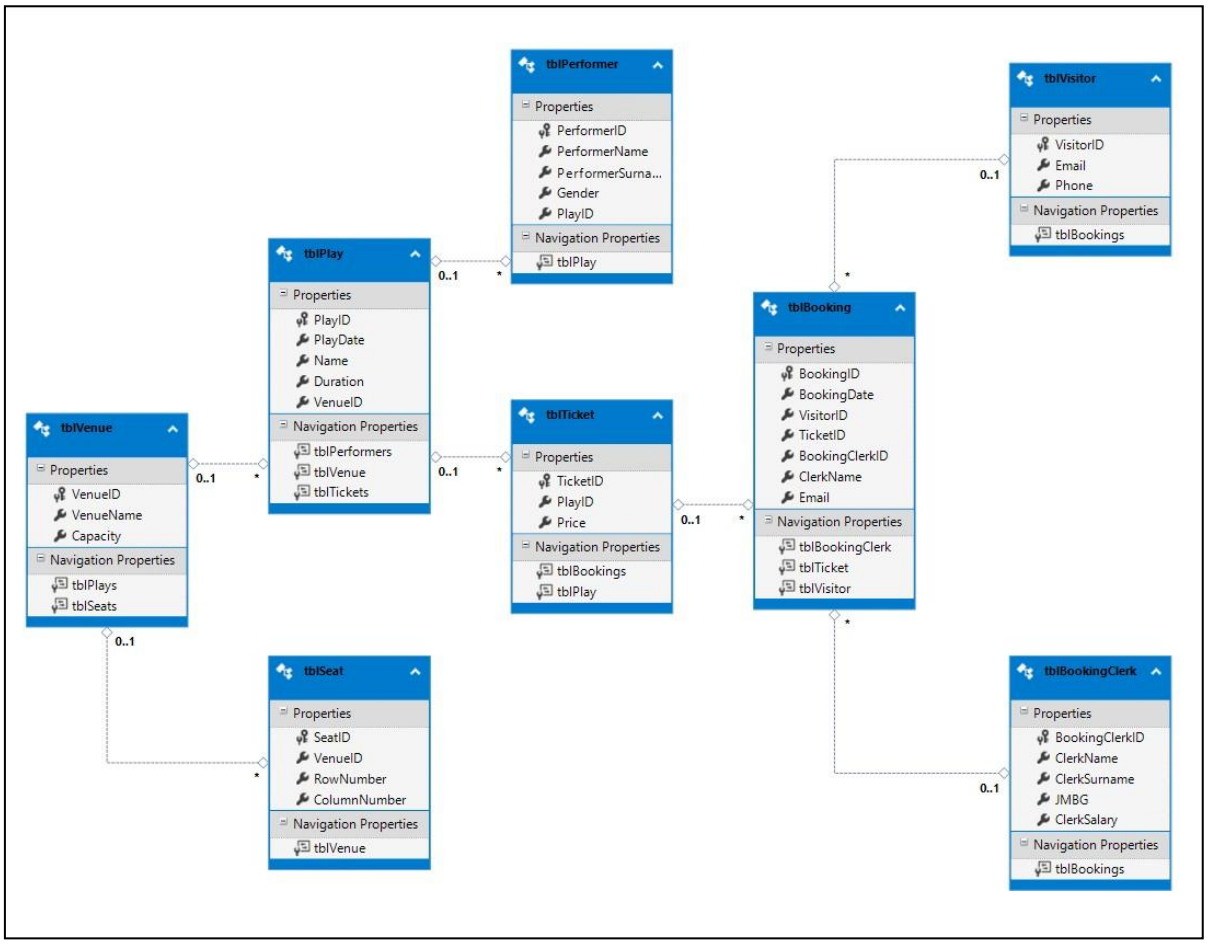
3.4. API

Application Programing Interface (API) predstavlja protokole i alate za kreiranje aplikacije, praktično definiše na koji način se ostvaruje interakcija između softverskih komponenata. Pristup API-ju zahteva proveru autentičnosti, zbog toga iz sigurnosnih razloga koristi API ključ. Takođe, definiše načine na koje aplikacija može da zahteva usluge od raznih biblioteka ili sistema, i poseduje rečnik i konvencije na koji način može da se primeni kako bi servis bio upotrebljen.

4. Prikaz rešenja

Kreiranje baze podataka je predstavljao prvi korak pri izradi aplikacije, upotrebljen je *SQL Server Management Studio* koji je u prethodnom poglavlju opisan. Baza podataka je potrebna radi lakšeg pretraživanja i pristupa podacima kao i sprovođenja metoda upisa, modifikacije i brisanja. Šema baze podataka *Theatre* (pozorište) kreirana za potrebe aplikacije i prikazana je na slici 1 se sastoji od 8 tabela a to su :

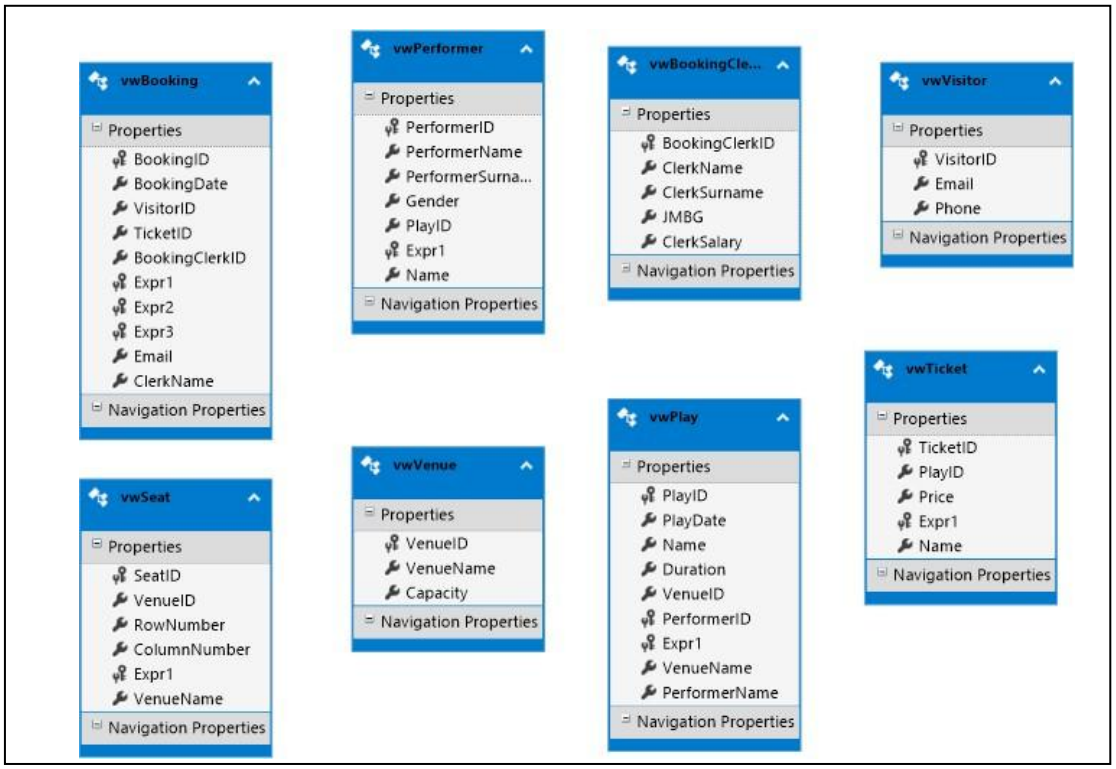
- *Booking* (rezervacija),
- *Booking Clerk* (blagajnik),
- *Performer* (izvođač),
- *Play* (predstava),
- *Seat* (sedište),
- *Ticket* (karta),
- *Venue* (sala) i
- *Visitor* (posetilac).



Slika 1. Šema baze podataka

Svaka tabela unutar baze podataka mora imati definisan jedinstveni identifikator, primarni ključ koji jednoznačno određuje tu tabelu i razlikuje je od preostalih. U okviru ove baze za svaku tabelu definisan je jedinstveni identifikator u formi „nazivtabeleID“. Za svaku navedenu tabelu kreirana su i obeležja, odnosno atributi koji detaljnije opisuju određenu tabelu i definisani su naziv, tip kao i da li određeni atribut može da ima *null* vrednost. Podešeno je da samo primarni ključ ne sme da ima *null* vrednost, dok za je preostale atribute

dozvoljena *null* vrednost kako ne bi dolazilo do problema prilikom upisa novih podataka u tabelu. Nakon kreiranja tabela potrebno je dodati relacije između navedenih entiteta, definisanje stranih ključeva kako bi se realni sistem u potpunosti prikazao, na ovaj način se povezuju podaci iz različitih tabela. Strani ključ mora da bude primarni ključ u nekoj drugoj tabeli kako bi mogao da referencira podatke iz željene tabele. Kreiranje pogleda je takođe izvršeno kako bi bio olakšan prikaz svih atributa iz određenih tabela, prikazano na slici 2.



Slika 2 Kreirani pogledi za potrebe aplikacije

4.1. WPF & WCF

S obzirom da ova WPF aplikacija putem Interneta treba da komunicira sa WCF servisom, naredni korak je bio kreiranje WCF servisa. U okviru *Visual Studio*-u je kreiran WCF servis koji je potrebno povezati sa prethodno kreiranom bazom podataka. Takođe i bazu podataka je potrebno dodati, kreiramo konekciju na željenu bazu i dodajemo prethodno spomenute tabele i poglede. WCF servis je dodat pomoću IIS Manager-a gde je kreiran naziv i localhost, kao i putanja do prethodno kreiranog servisa u *Visual Studio*-u.

U okviru WCF servisa, u klasi *Service1.svc.cs* su definisane za svaku tabelu pojedinačno metode za kreiranje liste podataka, upis, modifikaciju, brisanje i metoda za pozivanje pri selektovanju jednog reda u tabeli kao i metoda koja proverava da li se ključ te tabele nalazi u povezanoj tabeli, kako bi bilo omogućeno brisanje.

Implementacija prethodno navedenih metoda za tabelu *Booking* se nalazi na narednim slikama 2, 3, 4, 5 i 6. Za preostalih sedam tabela su na identičan način kreirane metode, iz tog razloga je prikazano samo za pojedinačnu tabelu implementacija metoda. Takođe za neke tabele je definisana i posebna metoda koja povezuje određeni ID sa nekim obeležjem kako bi se obezbedio prikaz.

Nakon definisanih metoda u klasi *Service1.svc.cs* potrebno ih je definisati i u *interface*-u *Iservice1.cs*.

```
        List<vwBooking> IService1.GetAllBooking()
        {
            try
            {
                using (TheatreEntities1 context = new TheatreEntities1())
                {
                    List<vwBooking> list = new List<vwBooking>();
                    list = (from x in context.vwBookings select x).ToList();
                    return list;
                }
            }
            catch (Exception ex)
            {
                System.Diagnostics.Debug.WriteLine("Exception" +
ex.Message.ToString());
                return null;
            }
        }
```

Slika 2. Metoda za kreiranje liste rezervacija

```
        vwBooking IService1.AddBookings(vwBooking booking)
        {
            try
            {
                using (TheatreEntities1 context = new TheatreEntities1())
                {
                    if (booking.BookingID == 0)
                    {
                        tblBooking newBooking = new tblBooking();
                        newBooking.BookingDate = booking.BookingDate;
                        newBooking.BookingClerkID =
ReturnBookingClerkID(booking.ClerkName);
                        newBooking.TicketID = booking.TicketID;
                        newBooking.VisitorID = ReturnVisitorID(booking.Email);
                        context.tblBookings.Add(newBooking);
                        context.SaveChanges();
                        booking.BookingID = newBooking.BookingID;
                        return booking;
                    }
                    else
                    {
                        tblBooking bookingToEdit = (from b in context.tblBookings
where b.BookingID == booking.BookingID select b).First();
                        bookingToEdit.BookingDate = booking.BookingDate;
                        bookingToEdit.BookingClerkID = booking.BookingClerkID;
                        bookingToEdit.TicketID = booking.TicketID;
                        bookingToEdit.VisitorID = booking.VisitorID;
                        bookingToEdit.BookingID = booking.BookingID;
                        context.SaveChanges();
                        return booking;
                    }
                }
            }
            catch (Exception ex)
            {
                System.Diagnostics.Debug.WriteLine("Exception" +
ex.Message.ToString());
                return null;
            }
```

Slika 3. Metoda za dodavanje/modifikaciju rezervacije

```
        void IService1.DeleteBooking(int bookingID)
        { try
            {
                using (TheatreEntities1 context = new TheatreEntities1())
                {
                    tblBooking bookingToDelete = (from b in context.tblBookings
where b.BookingID == bookingID select b).First();
                    context.tblBookings.Remove(bookingToDelete);
                    context.SaveChanges();
                }
            }
            catch (Exception ex)
            {
                System.Diagnostics.Debug.WriteLine("Exception" +
ex.Message.ToString());
            }
        }
```

Slika 4. Metoda za brisanje rezervacije

```
        List<vwBooking> IService1.GetBookingDetail(int visitorID)
        { try
            {
                using (TheatreEntities1 context = new TheatreEntities1())
                {
                    List<vwBooking> list = new List<vwBooking>();

                    list = (from x in context.vwBookings where x.VisitorID ==
visitorID select x).ToList();
                    return list;
                }
            }
            catch (Exception ex)
            {
                System.Diagnostics.Debug.WriteLine("Exception" +
ex.Message.ToString());
                return null;
            }
        }
```

Slika 5. Metoda za pozivanje pri selekciji reda

```
        vwBooking IService1.IsBookingID(int BookingID)
        {  try
            {
                using (TheatreEntities1 context = new TheatreEntities1())
                {
                    vwBooking b1 = new vwBooking();
                    b1 = (from x in context.vwBookings where x.BookingID ==
BookingID select x).FirstOrDefault();
                    return b1;
                }
            }
            catch (Exception ex)
            {
                System.Diagnostics.Debug.WriteLine("Exception" +
ex.Message.ToString());
                return null;
            }
          }
```

Slika 6. Metoda za omogućavanje brisanja

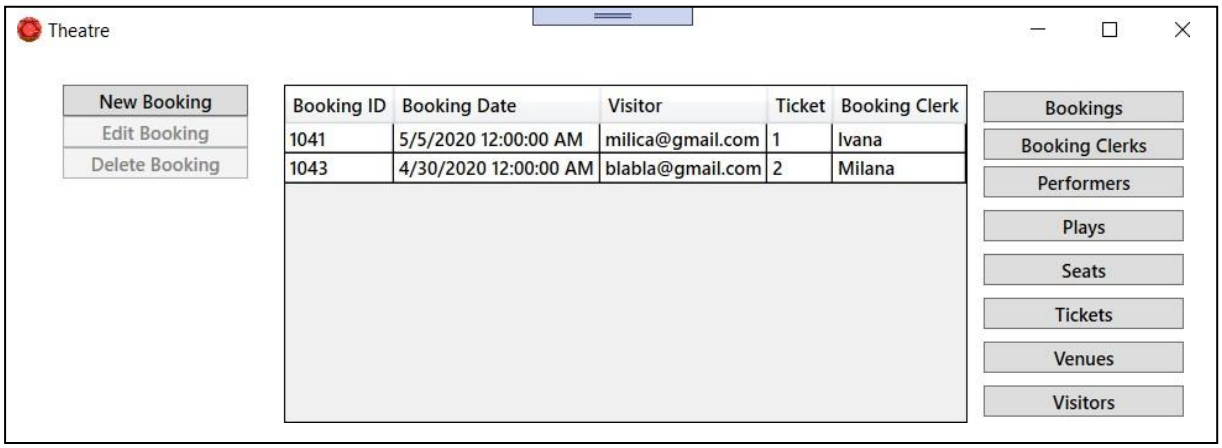
Naredni korak jeste kreiranje WPF aplikacije koja će komunicirati sa WCF servisom, preko servisne reference koja se dodaje. WPF predstavlja klijentsku stranu dok WCF predstavlja serversku stranu.

Klasa *RelayCommands* služi za odvajanje poslovne logike same komande uz pomoć delegata (referenci na metodu) , dok se u folderu *ViewModel* nalaze klase koje sadrže poslovnu logiku. Prvo se kreirana apstraktna metoda *ViewModelBase* koja kasnije proširuje preostale kreirane *ViewModele*-e za svaku tabelu ponaosob. Omogućava korišćenje metode *OnPropertyChanged* kako bismo ispratili svaku promenu na kontroli za koju je vezan taj *Propertie*.

MainWindowViewModel.cs se sastoji od tri regiona, u prvom *Constructor* su kreirana dva konstruktora, prvi pozivamo prilikom otvaranja *Main*-a dok drugi pozivamo prilikom *update*-ovanja sadržaja u tabeli. U regionu *Properties* su potrebni *Propertie*-i za svaku tabelu kao i za svaku listu. Takođe se tu podešava *Visibility* svakog *Propertie*-a, na početku su setovani na *Collapsed* dok se ne klikne na dugme, nakon toga postaju *Visible*.

Nakon toga imamo i treći region *Commands* gde definišemo koje komande se pozivaju prilikom klika na dugme i koje se metode izvršavaju npr. *AddNewBookingExecute* prilikom čega otvaramo formu *AddBooking*. Tu su definisane i preostale aktivinosti prilikom klika na dugme *Edit* izvršava se komanda *EditStudentExecute* gde prvo provervamo da li je selektovan uopšte red. Na kraju imamo dugme *Delete* i metodu *DeleteBookingExecute*, i tu pozivamo metodu *IsBookingID* prethodno definisanu u *Service1.svc.cs* gde se provera da li se taj ključ nalazi u drugoj tabeli ako se ne nalazi izvršava se brisanje u suprotno izlazi obaveštenje da nije moguće obrisati. *CanDeleteBookingExecute* onemogućava klik na dugme *Delete* ukoliko nije selektovan red.

U folderu *Forms* se nalaze sve kreirane windows forme za aplikaciju, glavna forma *MainWindow.xaml* kao i forma za svaku tabelu. Što se tiče glavne *windows* forme *MainWindow.xaml* koja se prikazuje pri pokretanju aplikacije se sastoji od *DataGrid*-ova za svaku tabelu, koji su *bind*-ovani za određene liste, u zavisnosti od toga koji *button* (dugme) je aktiviran ta lista podataka je prikazana. Takođe postoji *StackPanel* koji sadrži *Add, Edit* i *Delete button* za svaku tabelu. Prikaz pri pokretanju MainWindow-a je prikazan na slici 7.

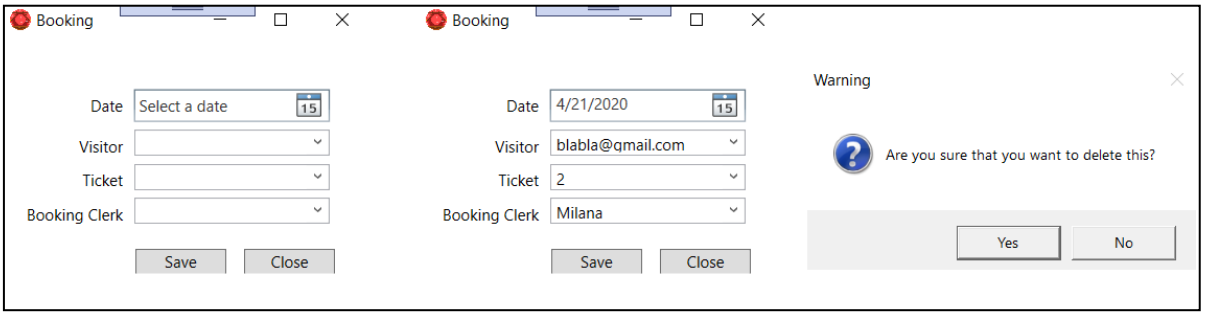


Slika 7. Izgled MainWindow.xaml

Prilikom klika na *Add* dugme otvara se forma za dodavanje novog rekorda u tabelu, dok je za aktiviranje dugmadi *Edit* i *Delete* potrebno prvo selektovati red u tabeli pa tek onda postoji mogućnost da izvršite modifikaciju i brisanje, prikazano na slici 8. Kod modifikacije se otvara ista forma kao za *Add* samo je u ovom slučaju popunjena podacima iz selektovanog reda i imate mogućnost da izmenite vrednost bilo kog atributa. Što se tiče *Delete* opcije pojavljuje se obaveštenje koje vas pita da li ste sigurni da želite da obrišete selektovani red, ukoliko potvrdno odgovorite rekord se iz baze briše a ukoliko odustanete rekord ostaje i dalje u bazi.

Slika 8. Edit i Delete dugme aktivno

Za svaku tabelu je kreirana *windows* forma ima *Code behind* koji opet služi za odvajanje poslovne logike od dizajna, gde imamo konstruktore kojima se prosleđuju uneti podaci u ovoj formi *ViewModel*-u za zadatu tabelu. U zavisnosti od obeležja i relacija sa drugim tabelama dodate su razne funkcionalnosti kao što je *ComboBox* koji služi kao padajuća lista u kojoj se vrši izlistavanje podataka iz neke druge tabele, *DataPicker* ukoliko je potreban unos datuma, *TextBox* za unos teksta kao i *button*-i *Save* i *Close*. Na slici 9 imamo prikaz *windows* forme za dodavanje nove rezervacije, modifikaciju postojeće i prikaz *MessageBox*-a koji proverava da li ste sigurni da želite da obrišete selektovan red.



Slika 9. Add, Edit i Delete Booking

4.2. Implementacija API-ja

Nakon izrade glavne forme aplikacije integrisana su naredna tri API-ja,:

1. FavQs API,
2. BingMaps API i
3. Weather API – OpenWeatherMap

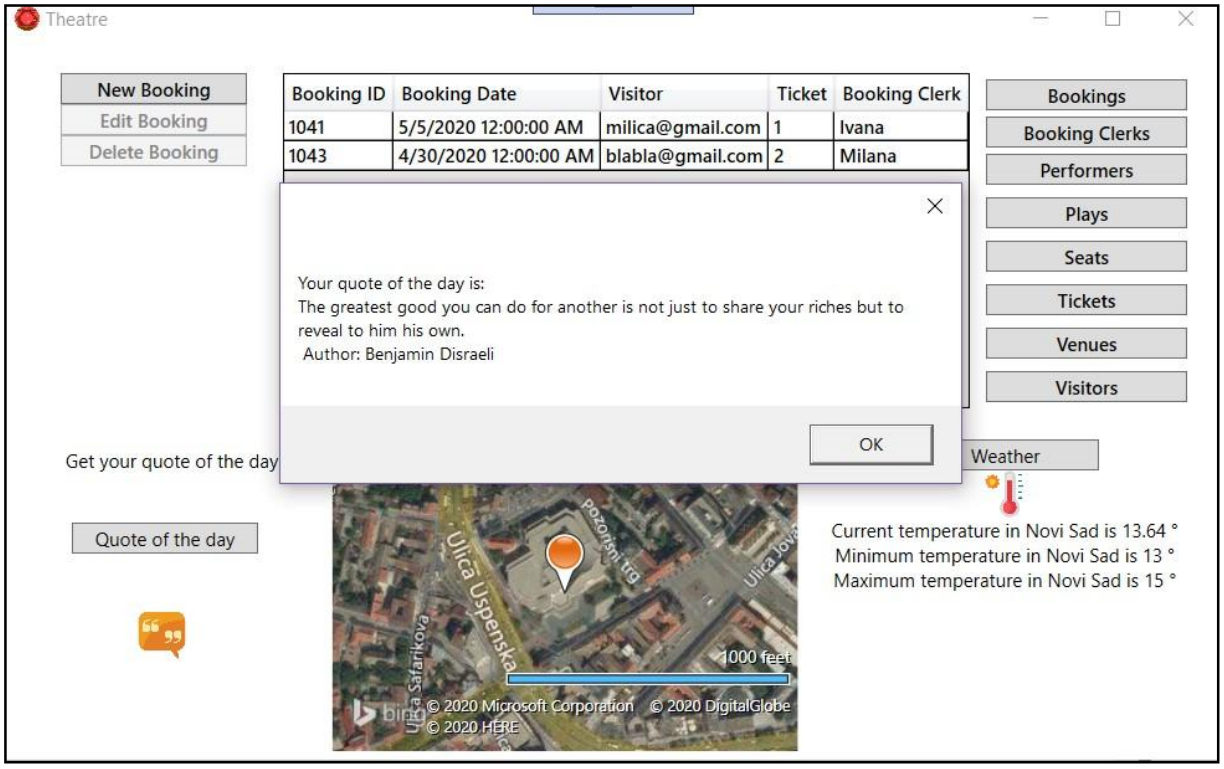
i za potrebe svakog od ova tri API-ja je kreiran API *key*, odnosno ključ koji nam pruža mogućnost da koristimo navedene servise u okviru zadate aplikacije.

Što se tiče prvog API-ja, **FavQs** API [3] koji sadrži omiljene citate, svi poslati i primljeni podaci su u JSON formatu. Preko HTTP zahteva je pristupljeno podacima, konvertovanje iz JSON formata je izvršeno kako bi bio omogućen ispis citata. Prilikom klika na dugme *Quote of the day* otvara se novi *MessageBox* koji prikazuje citat dana kao i ime autora tog konkretnog citata.

Naredni API jeste **BingMaps** API [4] pomoću kojeg je dodata mapa Novog Sada na glavnu formu, takođe dodat je i marker za lokaciju Srpskog narodnog pozorišta u Novom Sadu. Pomoću atributa Mode je definisano da će nam prikazi biti Aeral sa natpisima, odnosno nazivima ulica. Još jedna funkcionalnost je dodata na mapu, svaki put kada se klikne na neku lokaciju na mapi, pojavljuje se *MessageBox* u kom je ispisana tačna geografska širina (*latitude*) i geografska dužina (*longitude*) za lokaciju klika.

Na kraju je dodat i treći API, **Weather** API – **OpenWeatherMap** [5] pomoću kojeg pristupamo trenutnim vremenskim podacima za mnogobrojne lokacije u svetu. Za potrebe ove aplikacije je izabrana lokacija Novi Sad, i klikom na dugme *Weather* ispisuje se trenutna temperatura kao i minimalna i maksimalna temeperatura za taj dan. Potrebno je napomenuti da se veoma često ažurira baza sa podacima tako da su informacije ispisane na glavnom formi verodostojne u svakom trenutku.

Na slici 9 je prikazan izgled glavne forme sa integrisanim API-jima prethodno objašnjenim.



Slika 10. Izgled glavne forme sa integrisanim API-jima

5. Zaključak

Upotreba ovakve aplikacije bi olakšala i ubrzala mnoge procese što i jeste bio početni cilj izrade, uz dalji razvoj i modifikacije mogla bi da nađe još širu primenu. Uvid u evidencije o svim navedenim entitetima na jednom mestu i jednostavan korisnički interfejs su neke od prednosti ovog proizvoda.

S obzirom da je ova desktop aplikacija predviđena za korišćenje od strane zaposlenih u pozorištu neki od narednih koraka bi bio pravljenje veb aplikacije preko koje bi korisnici mogli samostalno da kreiraju svoje rezervacije.

Kako bi kreiranje rezervacija samostalno bilo omogućeno potrebno je izvršiti dodavanje entiteta u sadašnju bazu podataka. Podaci o režiseru, prikaz repertoara za naredni period, registracija posetilaca na sajt, prikaz dostupnih sedišta u određenom trenutku za određenu predstavu su neki od entiteta koji bi upotpunili ovaj informacioni sistem. Takođe implementacija dodatnih funkcionalnosti koje bi dozvoljavale elektronsko plaćanje karata ili automatsko slanje potvrde klijentu na e-mail o uspešno izvršenoj rezervaciji ali i API-ji koji omogućavaju pregled biblioteka, pretragu po naslovu knjige ili autora bi obogatili ovakvu aplikaciju.

6. Literatura

[1] Docs.microsoft.com. (2020.) *What Is Windows Communication Foundation - WCF*. Retrived from https://docs.microsoft.com/en-us/dotnet/framework/wcf/whats-wcf [9 April 2020].

[2] Docs.microsoft.com. (2020.) *WPF overview*. Retrived from https://docs.microsoft.com/en-us/dotnet/framework/wpf/introduction-to-wpf [9 April 2020].

[3] "FavQs API v2." *FavQs*, favqs.com/api/. [9 April 2020].

[4] Dev-Admin. "Documentation." *Bing Maps*, 12 Feb. 2020, www.microsoft.com/en-us/maps/documentation. [9 April 2020].

[5] OpenWeatherMap.org. "Current Weather Data." *Openweather*, openweathermap.org/current. [9 April 2020].